

“锁”之三



资源的分配策略

北京理工大学计算机学院
金旭亮

资源分配策略的公平性 (Fairness)



所谓“公平性”，是指资源的申请者是否按照其申请资源的顺序而被授予资源的独占使用权。



如果资源的任何一个先申请者总是能够比任何一个后申请者先获得该资源的独占权，那么，相应的资源调度策略就被称为是“**公平的**”。



如果资源的后申请者可能比先申请者先获得该资源的独占权，那么，相应的资源调度策略就被称为是“**不公平的 (Non-fair)**”。

"效率优先"的非公平策略



非公平策略允许插队，限制较少，因此可以比较灵活地安排各线程的访问资源的顺序。



非公平策略在极端情况下可能导致等待队列中的线程永远无法获得其所需的资源，即出现“饥饿（Starvation）”现象。



非公平策略的吞吐率较高。由于允许插队，它往往可以减少上下文切换的次数，获得较高的效率。



在资源的持有线程占用资源的时间相对较长，或线程申请资源的平均间隔时间较长的情况下，可以考虑使用公平调度策略。

分配策略的设定



在`ReentrantLock`类构造函数的参数中包含一个名为`fair`的`boolean`变量，开发者可以通过该变量为锁设置不同的策略模式。若值为`false`（这是默认设置）时则当前锁策略为非公平锁策略。

```
public ReentrantLock(boolean fair)
```



在非公平模式下，如果有多个线程同时请求同一个锁，那么该锁将随机分配给其中的一个线程。



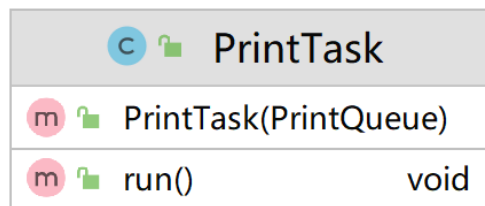
如果该策略设置为`true`，则表示当前策略为公平锁策略。在该策略下，如果当前有多个线程在等待同一把锁，则等待时间最长的线程将获得锁。

应用示例

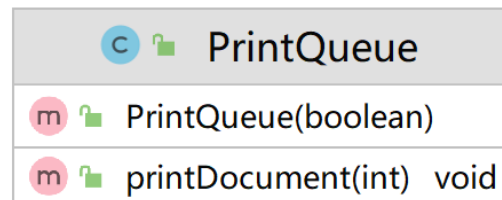


在Windows操作系统中打印文档，会有一个打印管理器，它将发往每台打印机的文档排成一个队列，顺序打印。

我们编写一个示例程序模拟这个过程：



打印任务



打印队列

打印任务

```
//一个虚拟的打印任务，每个任务打印两个文档
public class PrintTask implements Runnable {
    //引用打印机打印任务队列
    private final PrintQueue printQueue;
    public PrintTask(PrintQueue printQueue) {
        this.printQueue = printQueue;
    }
    @Override
    public void run() {
        var threadName :String = Thread.currentThread().getName();
        System.out.printf("%s: 将要开始打印两份文档\n", threadName);
        printQueue.printDocument( number: 1);
        printQueue.printDocument( number: 2);
        System.out.printf("%s: 文档已经全部打印完毕\n", threadName);
    }
}
```



打印任务（PrintTask）从外部接收一个打印队列（PrintQueue）实例，然后，调用它的printDocument方法打印文档。

打印队列

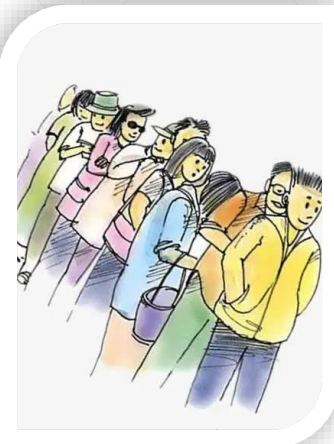
```
public class PrintQueue {  
    //互斥锁 (因为一台打印机一次只能打一个文档)  
    private final Lock queueLock;  
    public PrintQueue(boolean fairMode) {  
        queueLock = new ReentrantLock(fairMode);  
    }  
}
```

打印队列构造方法的参数，决定了要采用哪种资源分配策略。

由于打印机的物理特性，一次只能打印一个文档，所以，将打印过程放到临界区中。

//打印文档

```
public void printDocument(int number) {  
    queueLock.lock();  
    try {  
        long duration = (long) (Math.random() * 10000);  
        System.out.printf("%s: 打印第 %d 个文档, 耗时 %d 秒\n",  
            Thread.currentThread().getName(),  
            number,  
            (duration / 1000));  
        Thread.sleep(duration);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    } finally {  
        queueLock.unlock();  
    }  
}
```



公平策略下的输出



使用线程分配策略: `fair-mode = true`

文档打印者0: 将要开始打印两份文档
文档打印者0: 打印第 1 个文档, 耗时 8 秒
文档打印者1: 将要开始打印两份文档
文档打印者2: 将要开始打印两份文档
文档打印者3: 将要开始打印两份文档
文档打印者4: 将要开始打印两份文档
文档打印者5: 将要开始打印两份文档
文档打印者6: 将要开始打印两份文档
文档打印者7: 将要开始打印两份文档
文档打印者8: 将要开始打印两份文档
文档打印者9: 将要开始打印两份文档
文档打印者1: 打印第 1 个文档, 耗时 2 秒
文档打印者2: 打印第 1 个文档, 耗时 0 秒
文档打印者3: 打印第 1 个文档, 耗时 6 秒
文档打印者4: 打印第 1 个文档, 耗时 6 秒
文档打印者5: 打印第 1 个文档, 耗时 9 秒

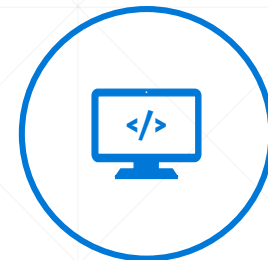
文档打印者6: 打印第 1 个文档, 耗时 8 秒
文档打印者7: 打印第 1 个文档, 耗时 7 秒
文档打印者8: 打印第 1 个文档, 耗时 4 秒
文档打印者9: 打印第 1 个文档, 耗时 1 秒
文档打印者0: 打印第 2 个文档, 耗时 7 秒
文档打印者0: 文档已经全部打印完毕
文档打印者1: 打印第 2 个文档, 耗时 2 秒
文档打印者1: 文档已经全部打印完毕
文档打印者2: 打印第 2 个文档, 耗时 2 秒
文档打印者2: 文档已经全部打印完毕
文档打印者3: 打印第 2 个文档, 耗时 1 秒
文档打印者3: 文档已经全部打印完毕
文档打印者4: 打印第 2 个文档, 耗时 1 秒
文档打印者4: 文档已经全部打印完毕
文档打印者5: 打印第 2 个文档, 耗时 3 秒
文档打印者5: 文档已经全部打印完毕
文档打印者6: 打印第 2 个文档, 耗时 5 秒
文档打印者6: 文档已经全部打印完毕

第一个请求锁的线程是Thread 0, 第二个为Thread 1, 以此类推。当Thread 0运行在第一段临界区代码时, 有其他九个线程等待在同一个临界区外。

Thread 0释放了锁的同时, 又再次发起对同一个锁的请求, 此时会同时有10个线程在请求同一把锁。基于公平锁策略, 锁的使用权将会交给等待时间最长的线程也就是Thread 1, 紧接着是Thread 2, 然后是Thread 3, 以此类推。

当所有线程都顺利通过第一个临界区却尚未进入第二个临界区时, 所有的线程都将在第二个临界区外部等待分配锁的使用权, 因此锁的使用权又重新交给了Thread 0线程, 待Thread 0执行完毕后交给Thread 1, 以此类推。

非公平策略下的输出



```
Run: Main x
"C:\Program Files\Java\jdk-17\bin\java.exe"
使用线程分配策略: fair-mode = false

文档打印者0: 将要开始打印两份文档
文档打印者0: 打印第 1 个文档, 耗时 8 秒
文档打印者1: 将要开始打印两份文档
文档打印者2: 将要开始打印两份文档
文档打印者3: 将要开始打印两份文档
文档打印者4: 将要开始打印两份文档
文档打印者5: 将要开始打印两份文档
文档打印者6: 将要开始打印两份文档
文档打印者7: 将要开始打印两份文档
文档打印者8: 将要开始打印两份文档
文档打印者9: 将要开始打印两份文档
文档打印者0: 打印第 2 个文档, 耗时 6 秒
文档打印者0: 文档已经全部打印完毕
文档打印者1: 打印第 1 个文档, 耗时 0 秒
文档打印者1: 打印第 2 个文档, 耗时 6 秒
文档打印者1: 文档已经全部打印完毕
文档打印者2: 打印第 1 个文档, 耗时 7 秒
```

当使用非公平策略时，各个线程是随机打印文档的，往往是一个线程先把两份文档打印完毕之后，另一个线程才有机会打印自己的文档。